QUẢN LÝ CONTEXT CHO AGENT – CHALLENGE B

Nhóm: 9
Môn học: SE373

1. Mục tiêu và công nghệ sử dụng

Bài thực hành triển khai hai kỹ thuật quản lý context nhằm giữ lượng dữ liệu gửi cho model nhỏ gọn nhưng vẫn bảo đảm agent thực hiện đúng nhiệm vụ:

- **B1 – Compaction:** rút gọn một phiên điều tra dài, giữ lại lỗi đang xử lý, các cách đã thử và bước tiếp theo.
- **B2 – Code Mode:** để model viết chương trình phân tích nhiều file log, xử lý dữ liệu trong runtime và chỉ nhận kết quả tổng hợp.

Ứng dụng sử dụng Python và thư viện chuẩn, gọi model thật thông qua Codex CLI. Prompt, phản hồi, chương trình do model sinh, kết quả công cụ và thông tin token được lưu trong thư mục evidence/.

Môi trường kiểm chứng: Python 3.9.6 và Codex CLI 0.164.0.

2. Cách hoạt động

2.1. B1 – Compaction

Chương trình tạo một lịch sử điều tra giả lập gồm nội dung trao đổi, kết quả thử nghiệm và nhiều bản ghi log. Khi dữ liệu phiên vượt ngưỡng 20.000 byte UTF-8, chương trình yêu cầu model tạo checkpoint có cấu trúc.

Checkpoint giữ lại mục tiêu, trạng thái hiện tại, ràng buộc, phát hiện quan trọng, các cách đã thử thất bại, những file liên quan và bước tiếp theo. Log thô và nội dung lặp được loại khỏi context đang hoạt động; lịch sử đầy đủ được lưu riêng để đối chiếu.

Sau đó, chương trình gửi checkpoint cho một lượt model mới để tiếp tục nhiệm vụ. Lượt này không nhận lại toàn bộ lịch sử cũ. Việc kiểm chứng tập trung vào khả năng xác định đúng lỗi chưa giải quyết, tránh lặp lại cách đã thất bại và tiếp tục đúng bước còn lại.

Ngưỡng kích hoạt được tính bằng byte UTF-8 của dữ liệu phiên; số input token được đo riêng từ kết quả gọi model. Việc compact do ứng dụng Python điều phối.

2.2. B2 – Code Mode

Nhiệm vụ là đọc tất cả file log, đếm các bản ghi có level bằng ERROR theo nhóm service/error_code và xác định ba nhóm xuất hiện nhiều nhất.

Chương trình so sánh hai cách xử lý trên cùng dữ liệu:

Cách xử lý	Luồng thực hiện
Đọc từng file	Model chọn hành động list_logs, lần lượt gọi read_log, nhận các bản ghi vào context rồi trả lời
Code Mode	Model viết code → Python chạy code trong runtime → runtime trả số liệu tổng hợp → model trả lời

Runtime cung cấp các hàm list_logs, read_log và emit. Code đọc, lọc và đếm dữ liệu bên trong runtime; chỉ kết quả emit được đưa về model.

Runtime diễn giải một tập con Python, giới hạn cú pháp, hàm được phép gọi, lượng dữ liệu, số bước và thời gian chạy. Code không được cung cấp công cụ shell, mạng hoặc quyền đọc đường dẫn tùy ý. Lỗi trong worker được chuẩn hóa thành loại lỗi và hướng dẫn cố định để tránh đưa nội dung log vào context qua exception.

Thông tin token được lấy từ turn.completed.usage.input_tokens của Codex CLI. Tổng input của mỗi phương pháp được tính bằng tổng tất cả lượt model trong phương pháp đó.

3. Kết quả kiểm chứng

3.1. Kết quả B1

Sau compact, agent vẫn xác định đúng:

- ISSUE-17 là lỗi refresh token chưa giải quyết; ISSUE-09 đã được xử lý.
- TRY-01: tăng TTL không khắc phục được lỗi.
- TRY-02: exponential backoff vẫn không khắc phục được lỗi.
- NEXT-03: kiểm tra vùng xử lý đọc/invalidate/rotate token và thiết kế atomic lock.
- Giữ các ràng buộc C-API về tương thích API và C-DEPS về việc không thêm dependency.

Chỉ số	Trước compact	Sau compact
Kích thước dữ liệu phiên gửi trong prompt	43.371 byte	1.571 byte
Input token của lượt tiếp tục	27.372	15.504

Input token của lượt tiếp tục giảm 43,36%.

Lượt tạo checkpoint sử dụng thêm 27.403 input token. Tổng của việc tạo checkpoint và tiếp tục là 42.907 input token. Vì vậy, kết quả này chứng minh lượt tiếp tục có context nhỏ hơn; chưa chứng minh compact tiết kiệm tổng token khi chỉ tiếp tục một lần.

3.2. Kết quả B2

Dữ liệu giả lập gồm 6 file log, 360 bản ghi, sử dụng seed 17. Cả hai phương pháp đều xác định đúng 74 bản ghi ERROR:

Nhóm lỗi	Số lượng
auth/REFRESH_RACE	47
search/CACHE_MISS	18
billing/DB_TIMEOUT	9
Tổng	**74**

Các bản ghi INFO và WARN không được tính.

Chỉ số	Đọc từng file	Code Mode
Số lượt model	8	2
Tổng input token	217.336	31.217

Code Mode giảm 85,64% tổng input token trong lần thử nghiệm này. Chương trình do model sinh thực hiện đúng phép đếm; câu trả lời cuối khớp với kết quả runtime và bộ kiểm chứng độc lập.

Hai prompt của Code Mode không chứa bản ghi log thô. Bộ kiểm chứng đối chiếu prompt với trace và chạy lại chương trình đã lưu để kiểm tra kết quả.

3.3. Kiểm thử tự động

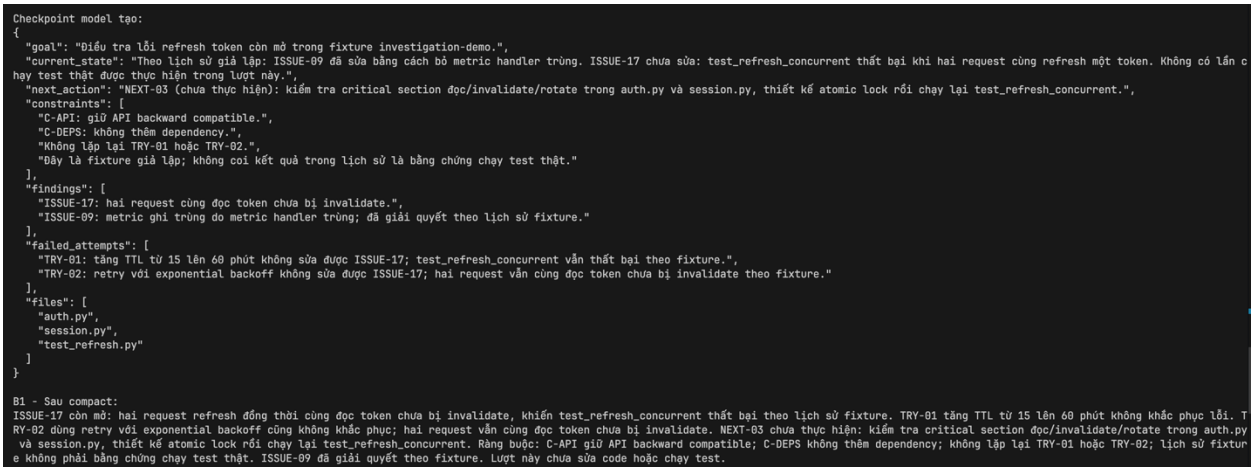
Bộ kiểm thử đạt 28/28, bao gồm compact, bảo toàn trạng thái, xử lý nhiều file, đối chiếu kết quả, giới hạn runtime, từ chối truy cập ngoài phạm vi và kiểm tra evidence.

Hai kiểm thử bổ sung xác nhận nội dung log không xuất hiện trong thông báo KeyError trả về và không bị đưa vào prompt của lượt sửa code tiếp theo. Các kiểm thử điều phối dùng model giả lập; bằng chứng hoạt động của model thật được kiểm tra riêng qua các lần chạy B1 và B2.

4. Minh chứng chạy chương trình

Hình 1 – Checkpoint và khả năng tiếp tục sau compact

Checkpoint giữ lại lỗi đang mở, các cách đã thử và bước tiếp theo. Câu trả lời sau compact sử dụng đúng những thông tin này.



Hình 2 – Chương trình do model sinh và kết quả runtime

Model viết code duyệt các file log, lọc bản ghi ERROR và tổng hợp số lượng theo nhóm. Runtime chỉ trả kết quả tổng hợp.

```
B2 - Chương trình model viết:
files = list_logs()
records_scanned = 0
counts = {}
for filename in files:
    records = read_log(filename)
    records_scanned += len(records)
    for record in records:
        if record["level"] == "ERROR":
            group = record["service"] + "/" + record["error_code"]
            counts[group] = counts.get(group, 0) + 1
emit({"files_scanned": len(files), "records_scanned": records_scanned, "counts": counts})

Runtime:
{
  "type": "tool_result",
  "ok": true,
  "result": {
    "files_scanned": 6,
    "records_scanned": 360,
    "error_counts": [
      {
        "group": "auth/REFRESH_RACE",
        "count": 47
      },
      {
        "group": "billing/DB_TIMEOUT",
        "count": 9
      },
      {
        "group": "search/CACHE_MISS",
        "count": 18
      }
    ]
  }
}
```

Hình 3 – So sánh input token

Kết quả thể hiện mức giảm input token của lượt tiếp tục B1 và tổng input token của B2.

```
B1 - INPUT TOKENS
{
  "before_input": 27372,
  "compact_creation_input": 27403,
  "after_input": 15504,
  "compact_path_total_input": 42907,
  "continuation_reduction_percent": 43.36
}

B1 - KIEM CHUNG
{
  "before_correct": true,
  "after_correct": true,
  "checkpoint_preserves_ids": true,
  "same_session_id": true,
  "raw_log_removed": true,
  "context_smaller": true,
  "continuation_input_tokens_smaller": true
}

B2 - INPUT TOKENS
{
  "baseline_input": 217336,
  "code_mode_total_input": 31217,
  "code_mode_final_input": 15760,
  "reduction_percent": 85.64
}

B2 - KIEM CHUNG
{
  "code_executed": true,
  "tool_correct": true,
  "final_matches_tool": true,
  "raw_log_absent_from_all_code_mode_prompts": true,
  "total_code_mode_input_tokens_smaller": true
}
```

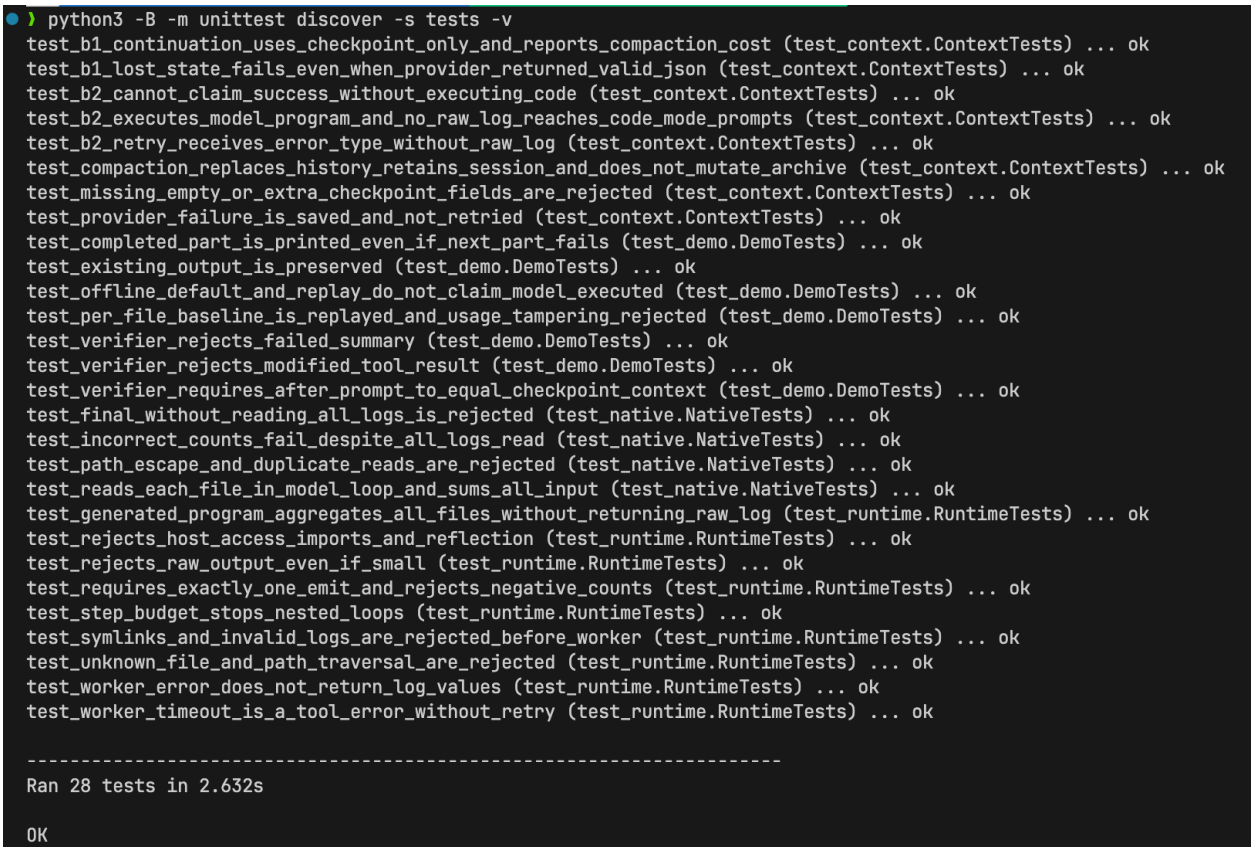
Hình 4 – Kiểm tra context của Code Mode

Các prompt Code Mode không chứa dấu nhận diện bản ghi log thô; kết quả công cụ và câu trả lời cuối đều vượt qua các bước đối chiếu.

```
CODE MODE - KIEM TRA LOG THO TRONG PROMPT
code-01.input.txt | raw log marker present: False
code-02.input.txt | raw log marker present: False
```

Hình 5 – Kết quả kiểm thử tự động

Bộ kiểm thử hoàn tất với 28 kiểm thử và trạng thái OK.



5. Kết luận

Trong kịch bản đã kiểm chứng, compaction giảm context của lượt tiếp tục và bảo toàn trạng thái điều tra cần thiết. Code Mode xử lý đúng dữ liệu trên nhiều file, giữ log thô trong runtime và giảm tổng input token so với cách model đọc từng file.

Dữ liệu điều tra và log là giả lập; checkpoint và chương trình trong lần chạy model thật do model tạo. Các tỷ lệ giảm token là kết quả của bộ dữ liệu thử nghiệm này. Runtime hiện phục vụ phạm vi bài thực hành, chưa phải sandbox Python tổng quát cho môi trường production.